SHETH LUJ AND SIR M.V. COLLEGE
PRACTICAL NO.:-1

AIM:-Implementing Substitution and Transposition Ciphers

1.A

Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

INPUT:-

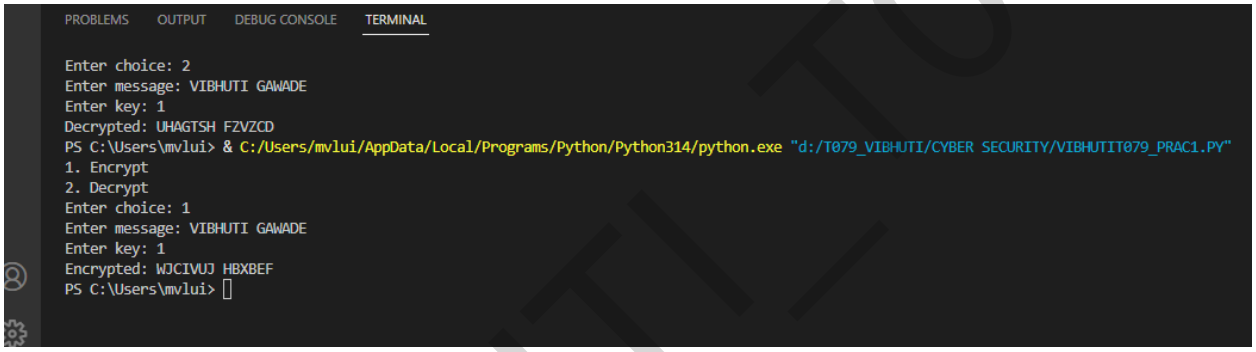
```
def encrypt(text, key):
    result = ""

    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch

    return result


def decrypt(text, key):
    result = ""

    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 - key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 - key) % 26 + 97)
        else:
            result += ch

    return result


print("1. Encrypt")
print("2. Decrypt")

choice = input("Enter choice: ")
```


VIBHUTI GAWADE
T079

SHETH LUJ AND SIR M.V. COLLEGE
PRACTICAL NO.:-1

```
text = input("Enter message: ")
key = int(input("Enter key: "))

if choice == '1':
    print("Encrypted:", encrypt(text, key))
elif choice == '2':
    print("Decrypted:", decrypt(text, key))
else:
    print("Invalid choice")
```

OUTPUT:-



1.A GUI
INPUT:-

```
import tkinter as tk
from tkinter import messagebox


def encrypt():
    try:
        text = entry.get()
        key = int(key_entry.get())

        result = ""

        for ch in text:
            if ch.isupper():
                result += chr((ord(ch) - 65 + key) % 26 + 65)
            elif ch.islower():
                result += chr((ord(ch) - 97 + key) % 26 + 97)
```

VIBHUTI GAWADE
T079

```
            else:
                result += ch

        output.config(text=result)

    except ValueError:
        messagebox.showerror("Error", "Please enter a valid numeric key")


def decrypt():
    try:
        text = entry.get()
        key = int(key_entry.get())

        result = ""

        for ch in text:
            if ch.isupper():
                result += chr((ord(ch) - 65 - key) % 26 + 65)
            elif ch.islower():
                result += chr((ord(ch) - 97 - key) % 26 + 97)
            else:
                result += ch

        output.config(text=result)

    except ValueError:
        messagebox.showerror("Error", "Please enter a valid numeric key")


root = tk.Tk()
root.title("Caesar Cipher")
root.geometry("500x400")
root.resizable(False, False)


root.configure(bg="#FFE4EC")

heading = tk.Label(
    root,
```

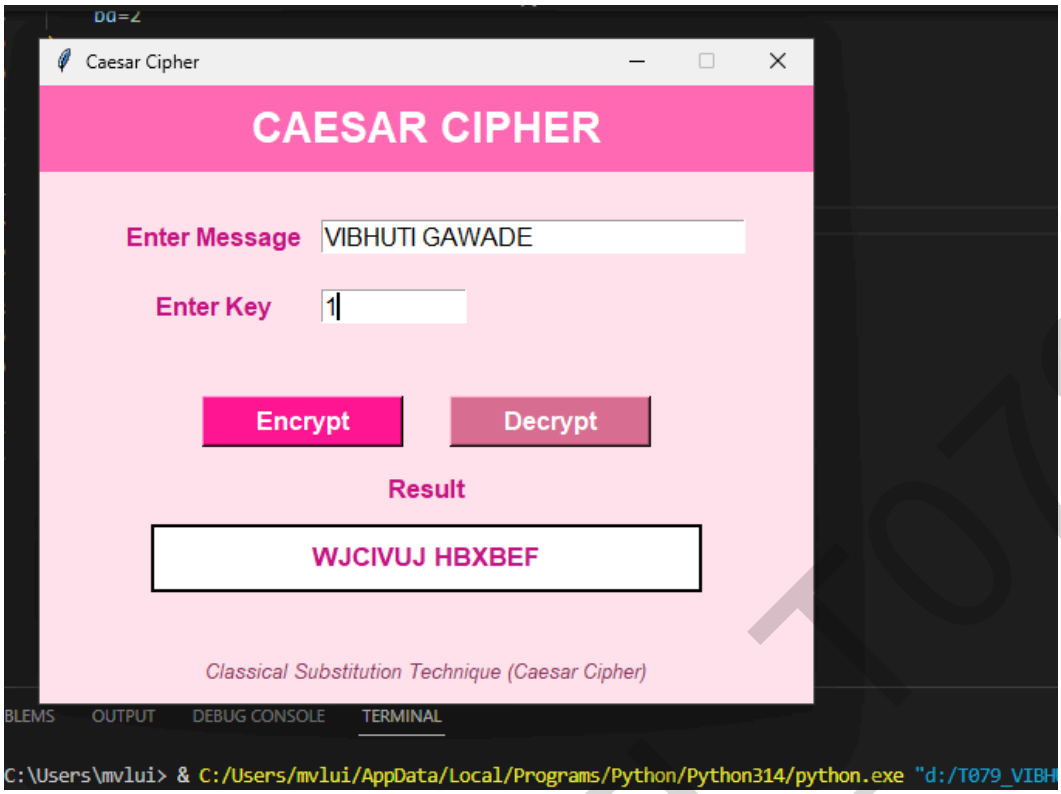
```
    text=" CAESAR CIPHER ",
    font=("Arial", 20, "bold"),
    bg="#FF69B4",
    fg="white",
    pady=10
)
heading.pack(fill="x")


frame = tk.Frame(root, bg="#FFE4EC")
frame.pack(pady=20)


tk.Label(
    frame,
    text="Enter Message",
    font=("Arial", 12, "bold"),
    bg="#FFE4EC",
    fg="#C71585"
).grid(row=0, column=0, padx=10, pady=10)

entry = tk.Entry(
    frame,
    width=30,
    font=("Arial", 12)
)
entry.grid(row=0, column=1)


tk.Label(
    frame,
    text="Enter Key",
    font=("Arial", 12, "bold"),
    bg="#FFE4EC",
    fg="#C71585"
).grid(row=1, column=0, padx=10, pady=10)

key_entry = tk.Entry(
    frame,
    width=10,
```

```
    font=("Arial", 12)
)
key_entry.grid(row=1, column=1, sticky="w")


button_frame = tk.Frame(root, bg="#FFE4EC")
button_frame.pack(pady=15)

encrypt_btn = tk.Button(
    button_frame,
    text="Encrypt",
    command=encrypt,
    bg="#FF1493",
    fg="white",
    font=("Arial", 12, "bold"),
    width=12,
    relief="raised"
)
encrypt_btn.grid(row=0, column=0, padx=15)

decrypt_btn = tk.Button(
    button_frame,
    text="Decrypt",
    command=decrypt,
    bg="#DB7093",
    fg="white",
    font=("Arial", 12, "bold"),
    width=12,
    relief="raised"
)
decrypt_btn.grid(row=0, column=1, padx=15)


tk.Label(
    root,
    text="Result",
    font=("Arial", 13, "bold"),
    bg="#FFE4EC",
    fg="#C71585"
).pack()
```

```
output = tk.Label(
    root,
    text="",
    bg="white",
    fg="#C71585",
    font=("Arial", 13, "bold"),
    width=35,
    height=2,
    relief="solid",
    bd=2
)
output.pack(pady=10)


footer = tk.Label(
    root,
    text="Classical Substitution Technique (Caesar Cipher)",
    font=("Arial", 10, "italic"),
    bg="#FFE4EC",
    fg="#8B3A62"
)
footer.pack(side="bottom", pady=10)


root.mainloop()
```

OUTPUT:-

SHETH LUJ AND SIR M.V. COLLEGE
PRACTICAL NO.:-1



VIBHUTI GAWADE
T079

SHETH LUJ AND SIR M.V. COLLEGE
PRACTICAL NO.:-1

1. B

Design and implement algorithms to encrypt and decrypt messages using transposition techniques

INPUT:-

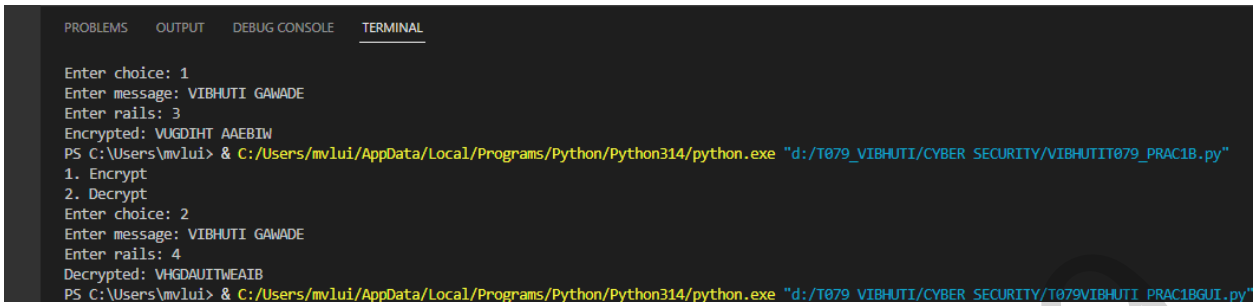
```
def encrypt(text, key):
    rail = [['\n' for i in range(len(text))]
            for j in range(key)]

    row, direction = 0, False

    for i in range(len(text)):
        if row == 0 or row == key - 1:
            direction = not direction

        rail[row][i] = text[i]
        row += 1 if direction else -1

    result = ""

    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                result += rail[i][j]

    return result


def decrypt(cipher, key):
    rail = [['\n' for i in range(len(cipher))]
            for j in range(key)]

    row, direction = 0, None

    for i in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False
```

```
        rail[row][i] = '*'
        row += 1 if direction else -1

    index = 0

    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row = 0

    for i in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False

        result += rail[row][i]
        row += 1 if direction else -1

    return result


print("1. Encrypt")
print("2. Decrypt")

choice = input("Enter choice: ")

text = input("Enter message: ")
key = int(input("Enter rails: "))

if choice == '1':
    print("Encrypted:", encrypt(text, key))
else:
    print("Decrypted:", decrypt(text, key))
```


VIBHUTI GAWADE
T079

SHETH LUJ AND SIR M.V. COLLEGE
PRACTICAL NO.:-1

OUTPUT:-



1.B GUI

INPUT:-

```
import tkinter as tk
from tkinter import messagebox

def encrypt(text, key):
    rail = [['\n' for i in range(len(text))]
            for j in range(key)]

    row, direction = 0, False

    for i in range(len(text)):
        if row == 0 or row == key - 1:
            direction = not direction

        rail[row][i] = text[i]
        row += 1 if direction else -1

    result = ""

    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                result += rail[i][j]

    return result


def decrypt(cipher, key):
    rail = [['\n' for i in range(len(cipher))]
            for j in range(key)]
```

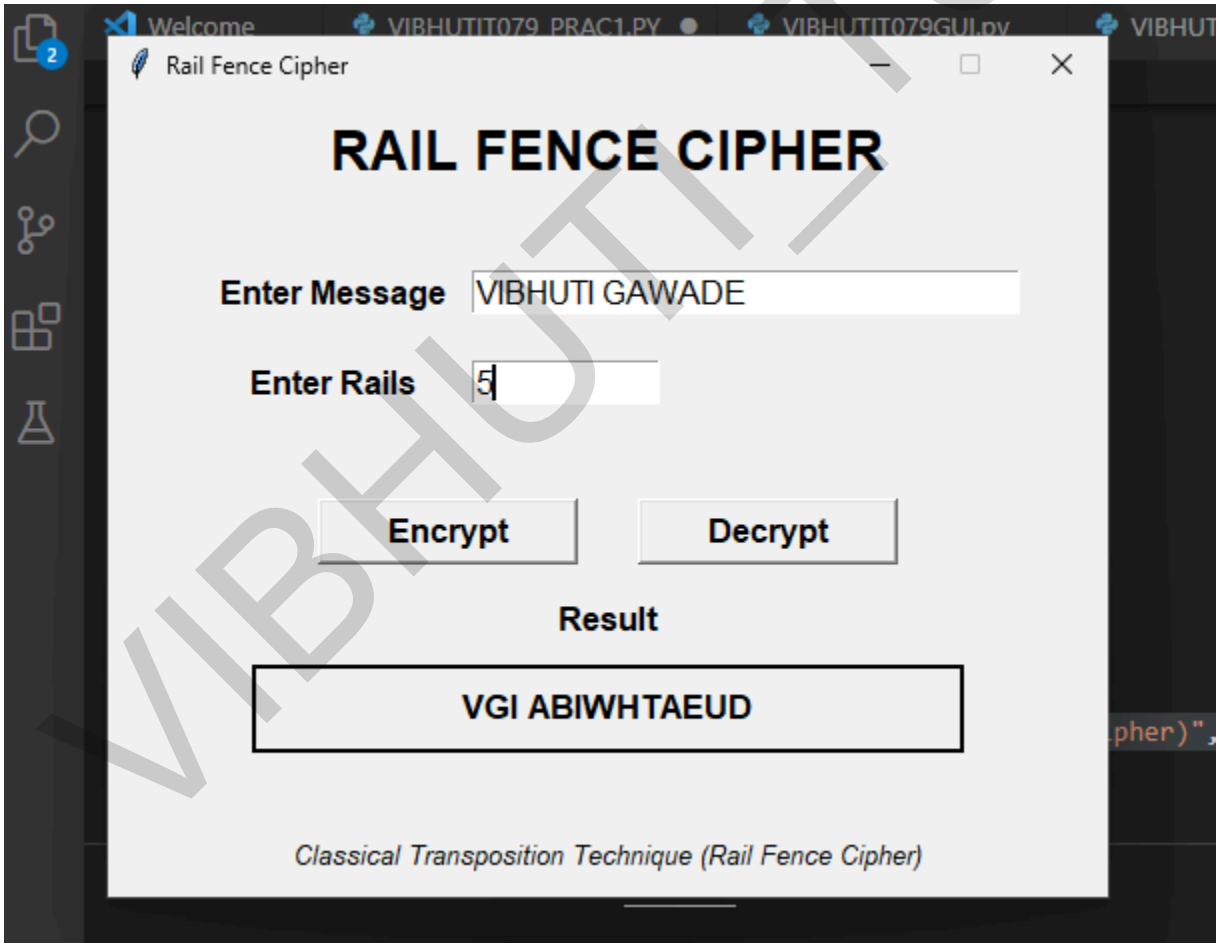
```
    row, direction = 0, None

    for i in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False

        rail[row][i] = '*'
        row += 1 if direction else -1

    index = 0

    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row = 0

    for i in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False

        result += rail[row][i]
        row += 1 if direction else -1

    return result


def encrypt_message():
    try:
        text = message_entry.get()
        key = int(key_entry.get())
        output.config(text=encrypt(text, key))
```

```
    except:
        messagebox.showerror("Error", "Enter a valid number of rails")


def decrypt_message():
    try:
        text = message_entry.get()
        key = int(key_entry.get())
        output.config(text=decrypt(text, key))
    except:
        messagebox.showerror("Error", "Enter a valid number of rails")


root = tk.Tk()
root.title("Rail Fence Cipher")
root.geometry("500x400")
root.resizable(False, False)

heading = tk.Label(
    root,
    text="RAIL FENCE CIPHER",
    font=("Arial", 20, "bold"),
    pady=10
)
heading.pack(fill="x")

frame = tk.Frame(root)
frame.pack(pady=20)

tk.Label(
    frame,
    text="Enter Message",
    font=("Arial", 12, "bold")
).grid(row=0, column=0, padx=10, pady=10)

message_entry = tk.Entry(frame, width=30, font=("Arial", 12))
message_entry.grid(row=0, column=1)

tk.Label(
    frame,
```

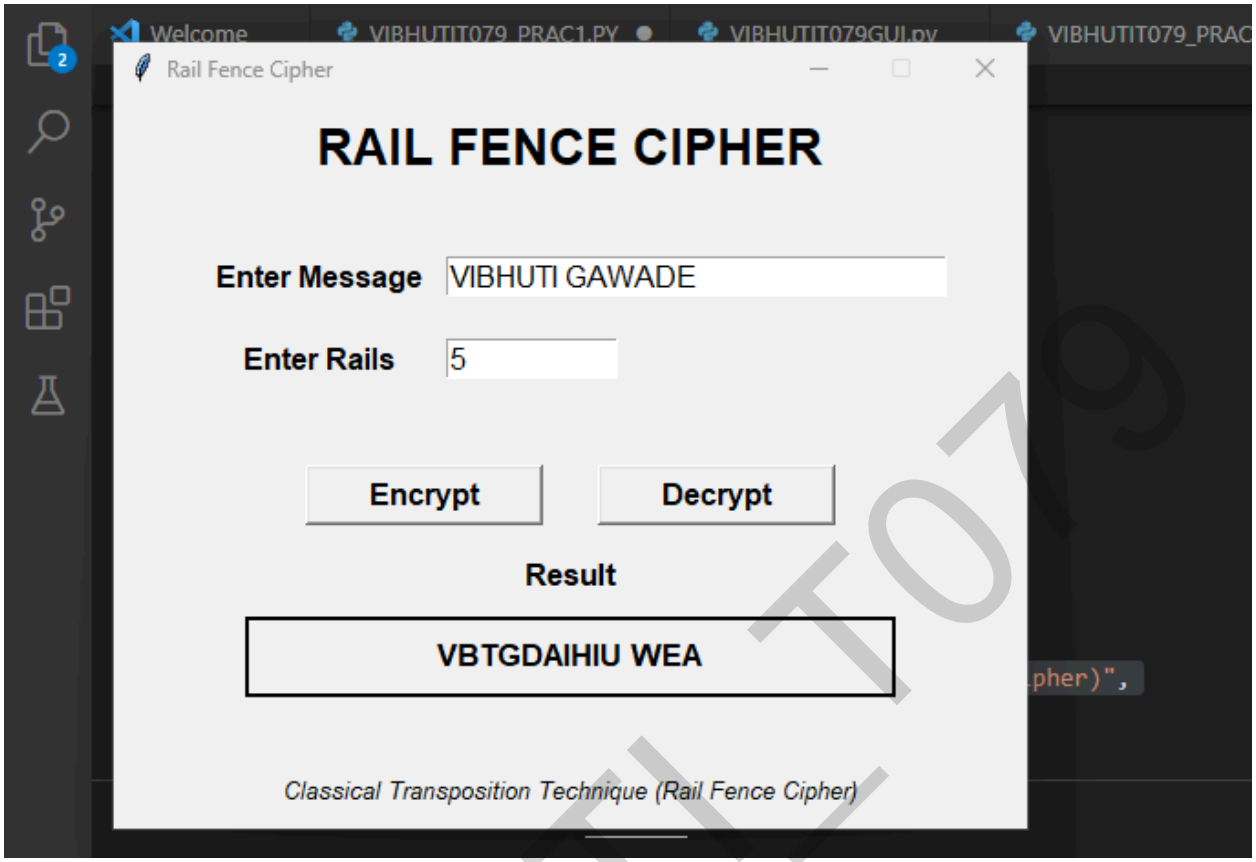
```
    text="Enter Rails",
    font=("Arial", 12, "bold")
).grid(row=1, column=0, padx=10, pady=10)

key_entry = tk.Entry(frame, width=10, font=("Arial", 12))
key_entry.grid(row=1, column=1, sticky="w")

button_frame = tk.Frame(root)
button_frame.pack(pady=15)

tk.Button(
    button_frame,
    text="Encrypt",
    command=encrypt_message,
    font=("Arial", 12, "bold"),
    width=12
).grid(row=0, column=0, padx=15)

tk.Button(
    button_frame,
    text="Decrypt",
    command=decrypt_message,
    font=("Arial", 12, "bold"),
    width=12
).grid(row=0, column=1, padx=15)

tk.Label(
    root,
    text="Result",
    font=("Arial", 13, "bold")
).pack()

output = tk.Label(
    root,
    text="",
    font=("Arial", 13, "bold"),
    width=35,
    height=2,
    relief="solid",
    bd=2
```

```
)
output.pack(pady=10)

footer = tk.Label(
    root,
    text="Classical Transposition Technique (Rail Fence Cipher)",
    font=("Arial", 10, "italic")
)
footer.pack(side="bottom", pady=10)

root.mainloop()
```

OUTPUT:-



SHETH LUJ AND SIR M.V. COLLEGE
PRACTICAL NO.:-1



VIBHUTI GAWADE
T079